

B2_Pulse_Perturbations

September 1, 2026

1 Bistable System: Pulse-Induced State Switching

```
[1]: from scipy.integrate import odeint
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, Dropdown
# (requires ipywidgets: pip install ipywidgets)

%matplotlib inline
# Use the lightweight static (inline) backend here, not an interactive one such
↳ as
# '%matplotlib widget'/ipympl -- those keep a persistent live canvas per figure
↳ and
# are the other common cause of the kernel freezing after repeated slider moves.
```

```
[2]: def forward_inhibition_open_model(variables, time, parameters):

    S, P = variables[0], variables[1]

    k_max, K_m, n, m = parameters[0], parameters[1], parameters[2],
↳ parameters[3]
    a_s, a_p, b_s, b_p = parameters[4], parameters[5], parameters[6],
↳ parameters[7]

    rate = k_max*S**n/(K_m + S**m)

    dS_dt = a_s - b_s*S - rate
    dP_dt = a_p - b_p*P + rate

    return [dS_dt, dP_dt]
```

```
[3]: # Bistable parameter set
# NOTE: in forward_inhibition_open_model the unpacking order is
#       a_s, a_p, b_s, b_p = parameters[4], parameters[5], parameters[6],
↳ parameters[7]
n, m = 1, 3
k_max, K_m = 2, 0.5
```

```

a_s, a_p, b_s, b_p = 1.2, 0.2, 0.3, 0.4
parameters = [k_max, K_m, n, m, a_s, a_p, b_s, b_p]

# The two stable steady states for this parameter set (found by locating the
# roots of  $a_s - b_s S - \text{rate}(S) = 0$  and checking their sign of  $df/dS$ ):
# low state:  $S^* \sim 0.35$ ,  $P^* \sim 3.74$ 
# high state:  $S^* \sim 4.31$ ,  $P^* \sim 0.77$ 
# with an unstable state in between at  $S^* \sim 1.24$  (the switching threshold)
S_low = [0.29, 3.28]
S_high = [3.45, 0.92]

# Pulse timing
pulse_start = 5
pulse_duration = 2
t_total = 40
time_step = 0.05

def simulate_with_pulse(amplitude, variables_init):
    """
    Integrate the model with a rectangular pulse added to  $a_s$  (i.e. a temporary
    extra influx/efflux of  $S$ ) of the given amplitude, applied between
    pulse_start and pulse_start + pulse_duration. Returns the full time course.
    """
    # before the pulse: normal  $a_s$ 
    t1 = np.arange(0, pulse_start, time_step)
    sol1 = odeint(forward_inhibition_open_model, variables_init, t1,
                  args=([k_max, K_m, n, m, a_s, a_p, b_s, b_p],))

    # during the pulse:  $a_s \rightarrow a_s + \text{amplitude}$ 
    t2 = np.arange(pulse_start, pulse_start + pulse_duration, time_step)
    sol2 = odeint(forward_inhibition_open_model, sol1[-1], t2,
                  args=([k_max, K_m, n, m, a_s + amplitude, a_p, b_s, b_p],))

    # after the pulse: back to the original  $a_s$ 
    t3 = np.arange(pulse_start + pulse_duration, t_total, time_step)
    sol3 = odeint(forward_inhibition_open_model, sol2[-1], t3,
                  args=([k_max, K_m, n, m, a_s, a_p, b_s, b_p],))

    time = np.concatenate([t1, t2, t3])
    variables = np.vstack([sol1, sol2, sol3])
    return time, variables

def plot_pulse_response(amplitude=1.0, start_state='low'):

```

```

plt.close('all') # drop any figure(s) from the previous call -- otherwise
↳they
                # pile up in memory on every slider move / cell re-run
↳and
                # eventually freeze the kernel

variables_init = S_low if start_state == 'low' else S_high
time, variables = simulate_with_pulse(amplitude, variables_init)

fig, ax = plt.subplots()

ax.axvspan(pulse_start, pulse_start + pulse_duration,
           color='grey', alpha=0.2, label='pulse')

ax.axhline(S_low[0], color='tomato', linestyle=':', linewidth=1)
ax.axhline(S_high[0], color='tomato', linestyle=':', linewidth=1)

ax.plot(time, variables[:, 0], c='tomato', label='S')
ax.plot(time, variables[:, 1], c='deepskyblue', label='P')

outcome = 'swapped' if abs(variables[-1, 0] - variables_init[0]) > 0.5 else
↳'returned'
ax.set_xlabel('time')
ax.set_ylabel('Amount')
ax.set_title(f'amplitude={amplitude:.2f}, start={start_state} -> 
↳{outcome}')
                f'(final S={variables[-1, 0]:.2f})')
ax.set_ylim(-0.1, 6)
ax.legend(loc='upper right')
plt.show()

# Slider re-runs the simulation and redraws the plot on every change.
# Roughly: |amplitude| > ~0.8 (from 'low') or > ~1.6 (from 'high') is enough to
↳swap state.
interact(plot_pulse_response,
        amplitude=FloatSlider(min=-1.5, max=2.5, step=0.1, value=1.0,
                               description='pulse amp.'),
        start_state=Dropdown(options=['low', 'high'], value='low',
                              description='start at'));

interactive(children=(FloatSlider(value=1.0, description='pulse amp.', max=2.5,
↳min=-1.5), Dropdown(descriptio...

```

Technical note — noise integration: The fluctuating term is added using the Euler–Maruyama scheme, the standard method for stochastic differential equations. The noise is scaled by $\sqrt{dt}$ (not dt), which correctly captures the statistics of random fluctuations in continuous time. See the in-code comments for details.

[]: